

Why Do Prompt Injection Attacks Succeed? Explaining Unauthorized Control Transfer in Tool-Using LLM Agents

Anonymous ACL submission

Abstract

Large Language Model (LLM) agents are vulnerable to prompt injection attacks, in which adversaries embed malicious instructions in untrusted tool outputs to redirect agents toward unauthorized objectives. Existing interpretability methods can detect anomalous signals associated with prompt injection attacks, but cannot determine whether the agents target action is driven by users and untrusted external instructions. In response, we investigate the causes of prompt injection attacks by explaining how users and untrusted external instructions influence LLM agents target actions. We observe unauthorized control transfer, in which injected instructions gained dominant control over the agents decision process. Building on this finding, an interpretable method with axiomatic guarantees is proposed, whereby user instructions are treated as the authorized-instruction region, untrusted external content is partitioned into two additional semantically distinct regionstask-relevant facts and injected instructionsand the marginal contributions of the resulting three regions to target actions are compared across successful and failed attacks. Experimental results on representative single-turn and multi-turn tool-use datasets reveal that unauthorized control transfer occurs when the contribution of injected instructions exceeds the combined contributions of authorized instructions and task-relevant facts. Our method improves accuracy by 47.02% and reduces the false-positive rate (FPR) by 92.46% relative to attention-based attribution. By explaining unsafe actions to authorized instructions, task-relevant facts, and injected instructions, our method supports post-attack forensics, security auditing, and mechanism-informed defense design.

1 Introduction

2 Methodology

Our framework provides an interpretable method for detecting unauthorized control transfer in tool-using language agents. As illustrated in Figure ??, the framework proceeds in two stages. First, Shapley attribution quantifies the contributions of the authorized-instruction, task-relevant facts, and injected instructions to the observed action, retaining cases in which injected instructions provide the dominant contribution. Second, an LLM-based behavior judge determines whether the observed action actually executes the injected instructions rather than simply mentioning or refusing them. An interaction is classified as a successful attack only when injected instructions dominate the attribution and its execution is behaviorally verified.

2.1 Unauthorized Control Transfer

Consider a tool-using LLM agent that receives an authorized instruction together with untrusted external content. Let x_{auth} denote the authorized instruction, which specifies the user task that the agent is expected to perform. Let x_{fact} and x_{attack} denote the tasks relevant facts and injected instructions contained in the untrusted external content, respectively. While x_{fact} provides information needed to complete the user task, x_{attack} attempts to steer the agent toward an attacker-specified task. The complete context is

$$x = (x_{\text{auth}}, x_{\text{fact}}, x_{\text{attack}}). \quad (1)$$

Given context x , the agent samples its next action y^* from

$$y^* \sim p_{\theta}(\cdot | x), \quad (2)$$

where y^* is either a natural-language response or a structured tool call. In the subsequent attribution

analysis, we hold y^* fixed as the explanatory target.

We define unauthorized control transfer (UCT) as a state in which x_{attack} exerts dominant influence over the agent’s decision, causing the resulting action to deviate from the user-specified goal and instead execute an attacker-specified task. Mere exposure or responsiveness to the injected instructions, such as acknowledging, mentioning, repeating, or refusing them, does not constitute UCT unless the resulting behavior is directed toward the attacker-specified task.

2.2 Shapley Attribution

Given the observed action y^* , we formulate a three-player cooperative game to quantify the contribution of each previously defined input region to the model’s support for this action. The player set is $N = \{\text{AUTH}, \text{FACT}, \text{ATTACK}\}$. Throughout the attribution analysis, y^* is held fixed across all coalitions, so that differences in coalition values reflect how the availability of different input regions changes the model’s conditional support for the same observed action.

For any coalition $S \subseteq N$, we retain the token embeddings of the player regions included in S and zero out the embeddings of those excluded from S . This embedding-level intervention preserves the original sequence length and token order because no tokens are removed or replaced, and the causal attention mask remains unchanged. All context outside the three player regions—including the system instructions, tool definitions, and message templates—is held fixed across coalition interventions. We denote the resulting input by x_S . For the empty coalition $S = \emptyset$, all three player regions are masked, while the remaining context is preserved. Thus, the coalition inputs differ only in which player regions contribute information when the model evaluates the fixed action y^* .

We define the coalition value $v(S)$ as the mean log-probability that the model assigns to the fixed observed action:

$$v(S) = \frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t^* | x_S), \quad (3)$$

where $y^* = (y_1^*, \dots, y_T^*)$ is the fixed observed action, y_t^* is the target token at output position t , and T is the number of output tokens in y^* . The term $p_{\theta}(y_t^* | x_S)$ is the conditional probability assigned

by the model to y_t^* under the coalition-specific input x_S . A larger $v(S)$ indicates stronger average conditional support for the fixed action y^* under coalition S .

For any player $i \in N$, its Shapley value is defined as

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)], \quad (4)$$

where ϕ_i is the attribution assigned to player i , and $v(S \cup \{i\}) - v(S)$ is its marginal contribution to coalition S . The Shapley value aggregates these marginal contributions over all possible coalitions, weighted according to the relative ordering in which player i can join the coalition.

To identify cases in which the injected instructions dominate the positive support from the legitimate regions, we define the following margin:

$$M_{\text{attack}} = \phi_{\text{ATTACK}} - [\max(0, \phi_{\text{AUTH}}) + \max(0, \phi_{\text{FACT}})], \quad (5)$$

where ϕ_{ATTACK} is the Shapley attribution of the injected instructions to the fixed action, while $\max(0, \phi_{\text{AUTH}})$ and $\max(0, \phi_{\text{FACT}})$ include only positive contributions from the AUTH and FACT regions. Thus, M_{attack} measures the extent to which the ATTACK contribution exceeds the sum of the positive contributions from the AUTH and FACT regions. We label y^* as an ATTACK-dominant candidate only when $M_{\text{attack}} > 0$.

2.3 Behavioral Verification

ATTACK dominance indicates only that the injected instructions provide the dominant positive support for the fixed observed action y^* , but it does not determine whether the action actually follows, merely refers to, or explicitly rejects the injected instructions. This distinction is important because an action that refuses, paraphrases, or discusses the injected content may still exhibit a strong attribution to the ATTACK region without constituting a successful attack. We therefore subject all samples satisfying $M_{\text{attack}} > 0$ to behavioral verification by an LLM-based judge.

The LLM-based judge receives a structured input record containing (i) the authorized task x_{auth} , (ii) the complete tool response, (iii) the identified injected instructions x_{attack} , (iv) the fixed observed action y^* , (v) the available tool definitions, and (vi) feedback from the external environment e . The feedback e captures observable post-action

evidence from the external environment. Based on this information, the judge assesses whether the observed action y^* behaviorally aligns with the injected instructions x_{attack} in terms of the intended target, operation type, and key parameters.

The LLM-based judge assigns the observed action y^* a binary behavioral label:

$$J(x_{\text{auth}}, x_{\text{attack}}, y^*, e) \in \{0, 1\}, \quad (6)$$

where $J(\cdot) = 1$ indicates that y^* behaviorally aligns with the injected instructions in terms of the intended target, operation type, and key parameters, while $J(\cdot) = 0$ indicates that y^* only mentions, restates, or rejects the injected instructions without carrying them out.

Strict attack success requires both attributional dominance and behavioral execution: the injected instructions must provide the dominant positive support for the fixed action, and the observed action must behaviorally follow those instructions. Formally,

$$M_{\text{attack}} > 0 \quad \text{and} \quad J = 1, \quad (7)$$

The first condition is attributional: $M_{\text{attack}} > 0$ indicates that the ATTACK attribution exceeds the combined positive attribution of the AUTH and FACT regions. The second condition is behavioral: $J = 1$ requires the observed action to align with the injected instructions in terms of the intended target, operation type, and key parameters. A sample is classified as a strict attack success only when both conditions hold. Together, these conditions exclude cases in which the observed action is not predominantly attributable to ATTACK, as well as cases that merely mention, restate, or reject the injected instructions without carrying them out.

References

Limitations

This is an Limitations

3 Example Appendix

This is an appendix.